



Part 2: Reflective Journal

Personal Reflection on Performance Analysis and CNN Validation

Author: Olu



Before completing this laboratory on Convolutional Neural Networks, my understanding of model evaluation was largely superficial. I understood that training accuracy and validation accuracy were important metrics to monitor, but I was less confident about interpreting them together in a meaningful way. When looking at preliminary outputs, it was easy to see a high accuracy value and immediately assume that the model was successful. However, this lab reconstructed my perspective, showing me that performance analysis requires examining training behavior, validation behavior, and final test results as interconnected parts of a cohesive learning process.

One of the most profound learning insights I gained from our lab was how deeply interconnected the preprocessing, architectural, and evaluation phases are. I initially viewed these as separate, isolated steps. However, by building our specific MNIST classifier, I saw how normalization (scaling pixels from a 0–255 range to a 0.0–1.0 range) and one-hot encoding directly dictated the effectiveness of the Adam optimizer and the categorical crossentropy loss function. My understanding of neural networks deepened when examining our specific architecture: seeing the progression from two Conv2D layers (using 32 and 64 filters to learn edges and complex patterns, respectively) through MaxPooling layers, and finally to a Dense softmax output, reminded me that an accuracy value is the outcome of a sequential feature-extraction process, not a standalone explanation. It was particularly surprising to see just how effective the 50% Dropout layer was in preventing the network from simply memorizing the 60,000 training images.

My primary challenge during this implementation was learning to distinguish a mathematically impressive result from one that is genuinely supported by evidence of generalization. When our model achieved a 99.07% test accuracy, my instinct was to simply declare the experiment a complete success. However, I struggled to articulate *why* it generalized well. I overcame this challenge by actively engaging with the 10% validation split checkpoint implemented in our lab. By comparing the validation accuracy against the training accuracy across our 15 epochs, I learned to look for the divergence point that signals overfitting. A strategy that helped me grasp this difficult concept was charting the conceptual pipeline of our architecture, moving away from code syntax and focusing purely on the mechanical flow of data.

This lab fundamentally changed my understanding of deep learning by proving that model building is a highly interpretive practice rather than a rigid formula. Choosing a batch size of 128 was not just about maximizing a single metric; it involved balancing the computational cost of 469 learning updates per epoch with the need for stable weight adjustments. I now recognize that because MNIST is a relatively controlled, single-channel

grayscale task, our strong performance does not necessarily establish that this exact architecture would perform equally well on a harder, unconstrained image dataset. Moving forward, I would like to explore how these foundational CNN mechanics change when applied to multi-channel color images or real-world, noisy datasets. This laboratory guided me from a place of metric-driven uncertainty to a solid, mechanical foundation. I now possess a structured framework for reading training results, diagnosing potential generalization problems, and critically explaining the underlying reasons behind a neural network’s performance.

Architectural Stage	Mechanical Absolute and Future Application
Dimensional Normalization	Constrains the raw 28x28 matrix into a uniform [0.0, 1.0] mathematical domain to stabilize gradient descent. Further exploration demands adapting these scaling boundaries for multi-channel or high-variance sensor inputs.
Hierarchical Feature Extraction	Utilizes localized Conv2D filters (32 and 64) and spatial compression (MaxPooling) to isolate geometric abstractions. Future scaling requires evaluating how filter depth and kernel sizing adapt to less controlled, asymmetrical image sets.
Stochastic Regularization	Actively severs 50% of neural pathways during training to prevent dataset memorization and force robust, distributed pattern recognition. Next steps involve testing dynamic or layer-specific dropout thresholds in deeper networks.
Probabilistic Classification	Translates flattened high-dimensional features into a normalized 10-class probability vector, driving the categorical crossentropy loss calculation. Exploring beyond MNIST involves adapting this terminal layer for multi-label tasks or continuous regression.

Table 1: Absolute Mechanical Breakdown and Exploratory Trajectory of the CNN Architecture